

МИНОБРНАУКИ РОССИИ
федеральное государственное бюджетное образовательное учреждение
высшего образования
«Балтийский государственный технический университет «ВОЕНМЕХ» им. Д.Ф. Устинова»
(БГТУ «ВОЕНМЕХ» им. Д.Ф. Устинова)

Кафедра	<u>О7</u>	<u>Информационные системы и программная инженерия</u>
	шифр	наименование кафедры, по которой выполняется работа
Дисциплина	<u>Разработка пользовательского интерфейса</u>	
		наименование дисциплины

ПРАКТИЧЕСКАЯ РАБОТА

4

номер задания (при наличии)

ИСПОЛЬЗОВАНИЕ CUDA

Вариант 12

при наличии указать тему учебно-практической работы и (или) номер варианта

ОБУЧАЮЩИЙСЯ

группы О717Б

Праудин Д.С.

подпись

фамилия и инициалы

дата сдачи

ПРОВЕРИЛ

ученая степень, ученое звание, должность

Лестенко Н.А.

подпись

фамилия и инициалы

Оценка / балльная оценка

дата проверки

Санкт-Петербург

20 24 г.

СОДЕРЖАНИЕ

1 Постановка задачи	4
2 Реализация	5
3 Результаты тестирования	10
ЗАКЛЮЧЕНИЕ	10

1 Постановка задачи

1. необходимо реализовать два этапа вашей программы из предыдущих лабораторных работ. При этом вычисления можно проводить как на CPU, так и на GPU (на своё усмотрение, но GPU предпочтительнее);
2. произвести расчет доверительного интервала;
3. посчитать время двумя способами: с помощью profiling и с помощью обычного замера (как в предыдущих заданиях);
4. оценить накладные расходы, такие как доля времени, проводимого на каждом этапе вычисления («нормированная диаграмма с областями и накоплением»), число строк кода, добавленных при распараллеливании, а также грубая оценка времени, потраченного на распараллеливание (накладные расходы программиста), и т.п.;
5. при желании данную лабораторную работу можно написать на CUDA.

2 Реализация

Lab4.c:

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <sys/time.h>

#define N1 1000
#define N2 10000
#define A 100

__global__ void processArrays(float *M1, float *M2, int N) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < N) {
        M1[idx] = sinh(M1[idx]) * sinh(M1[idx]);
    }
    if (idx < N / 2) {
        M2[idx] = fabs(sin(M2[idx]));
    }
}

void generateArrays(float *M1, float *M2, int N) {
    unsigned int seed = 0;
    for (int i = 0; i < N; i++) {
        M1[i] = 1 + (float)rand_r(&seed) / RAND_MAX * (A - 1);
    }
    for (int i = 0; i < N / 2; i++) {
        M2[i] = A + (float)rand_r(&seed) / RAND_MAX * (10 * A - A);
    }
}

void runCUDA(float *M1, float *M2, int N) {
    float *d_M1, *d_M2;
    cudaMalloc((void**)&d_M1, N * sizeof(float));
    cudaMalloc((void**)&d_M2, (N / 2) * sizeof(float));
```

```

    cudaMemcpy(d_M1, M1, N * sizeof(float), cudaMemcpyHostToDevice);
    cudaMemcpy(d_M2, M2, (N / 2) * sizeof(float), cudaMemcpyHostToDevice);

    int threadsPerBlock = 256;
    int blocksPerGrid = (N + threadsPerBlock - 1) / threadsPerBlock;

    processArrays<<<blocksPerGrid, threadsPerBlock>>>(d_M1, d_M2, N);

    cudaMemcpy(M1, d_M1, N * sizeof(float), cudaMemcpyDeviceToHost);
    cudaMemcpy(M2, d_M2, (N / 2) * sizeof(float), cudaMemcpyDeviceToHost);

    cudaFree(d_M1);
    cudaFree(d_M2);
}

void mapArrays(float *M1, float *M2, int N) {
    for (int i = 0; i < N; i++) {
        M1[i] = sinh(M1[i]) * sinh(M1[i]);
    }
    float *M2_copy = (float *)malloc((N / 2) * sizeof(float));
    for (int i = 0; i < N / 2; i++) {
        M2_copy[i] = M2[i];
    }
    for (int i = 1; i < N / 2; i++) {
        M2[i] += M2_copy[i - 1];
        M2[i] = fabsf(sinf(M2[i]));
    }
    free(M2_copy);
}

void mergeArrays(float *M1, float *M2, int N) {
    for (int i = 0; i < N / 2; i++) {
        M2[i] = powf(M1[i], M2[i]);
    }
}

void combSort(float *arr, int n) {
    int gap = n;

```

```

int swapped = 1;
while (gap != 1 || swapped == 1) {
    gap = (gap * 10) / 13;
    if (gap < 1)
        gap = 1;
    swapped = 0;
    for (int i = 0; i < n - gap; i++) {
        if (arr[i] > arr[i + gap]) {
            float temp = arr[i];
            arr[i] = arr[i + gap];
            arr[i + gap] = temp;
            swapped = 1;
        }
    }
}
}

```

```

float reduceArray(float *M2, int N) {
    float min = M2[0];
    for (int i = 1; i < N / 2; i++) {
        if (M2[i] < min && M2[i] != 0) {
            min = M2[i];
        }
    }
    float sum = 0;
    for (int i = 0; i < N / 2; i++) {
        if ((int)(M2[i] / min) % 2 == 0) {
            sum += sinf(M2[i]);
        }
    }
    return sum;
}

```

```

int main() {
    int N = N1; // Определяем размер массивов

    float *M1 = (float *)malloc(N * sizeof(float));
    float *M2 = (float *)malloc((N / 2) * sizeof(float));
}

```

```

struct timeval T1, T2;
long delta_ms;
float result;
long total_ms = 0;
float* iteration_times = (float*)malloc(100 * sizeof(float));

for (int i = 0; i < 100; i++) {
    gettimeofday(&T1, NULL);

    srand(i);
    generateArrays(M1, M2, N);
    mapArrays(M1, M2, N);
    mergeArrays(M1, M2, N);
    combSort(M2, N / 2);
    result = reduceArray(M2, N);

    gettimeofday(&T2, NULL);
    delta_ms = (T2.tv_sec - T1.tv_sec) * 1000 + (T2.tv_usec - T1.tv_usec)
/ 1000;
    total_ms += delta_ms;
    iteration_times[i] = delta_ms;

    printf("Iteration %d: X = %f, Milliseconds passed: %ld\n", i, result,
delta_ms);
}

printf("Total time for all iterations: %ld milliseconds\n", total_ms);

// Рассчитываем среднее время выполнения итераций
float mean_time = total_ms / 100.0;

// Рассчитываем стандартное отклонение
float sum_squared_diff = 0;
for (int i = 0; i < 100; i++) {
    sum_squared_diff += pow(iteration_times[i] - mean_time, 2);
}
float std_dev = sqrt(sum_squared_diff / 100);

```

```
// Рассчитываем доверительный интервал (95%)
float t_value = 1.984; // Для 99 степеней свободы
float margin_of_error = t_value * (std_dev / sqrt(100));

// Выводим результаты
printf("Mean time: %.2f milliseconds\n", mean_time);
printf("Standard deviation: %.2f milliseconds\n", std_dev);
printf("Confidence interval (95%%): %.2f +- %.2f milliseconds\n",
mean_time, margin_of_error);

free(M1);
free(M2);
free(iteration_times);

return 0;
}
```


3 Результаты тестирования

Изменение программы под CUDA, представлено в заголовки реализации. Результат работы программы продемонстрирован на рисунке 1.

```
Iteration 95: X = -nan, Milliseconds passed: 0
Iteration 96: X = -nan, Milliseconds passed: 0
Iteration 97: X = -nan, Milliseconds passed: 0
Iteration 98: X = -nan, Milliseconds passed: 0
Iteration 99: X = -nan, Milliseconds passed: 0
Total time for all iterations: 0 milliseconds
```

Рисунок 1 – Результат работы программы сделанной под CUDA

Программа предусматривает расчёт доверительного интервала, который продемонстрирован на рисунке 2.

```
Iteration 95: X = -nan, Milliseconds passed: 0
Iteration 96: X = -nan, Milliseconds passed: 0
Iteration 97: X = -nan, Milliseconds passed: 0
Iteration 98: X = -nan, Milliseconds passed: 0
Iteration 99: X = -nan, Milliseconds passed: 0
Total time for all iterations: 0 milliseconds
Mean time: 0.00 milliseconds
Standard deviation: 0.00 milliseconds
Confidence interval (95%): 0.00 +- 0.00 milliseconds
```

Рисунок 2 – Результат работы программы, расчёт доверительного интервала

Чтобы рассчитать доверительный интервал для среднего времени выполнения итераций, можно воспользоваться формулой для расчета доверительного интервала для среднего значения выборки. Допустим, у нас есть время выполнения каждой итерации, и мы хотим рассчитать доверительный интервал с уровнем доверия, например, 95%, для этого необходимо:

1. вычислите среднее время выполнения итераций;
2. вычислите стандартное отклонение времени выполнения итераций;

3. найдите критическое значение t -статистики для выбранного уровня доверия (например, 95%) и количества наблюдений (в данном случае, количество итераций);

4. вычислите стандартную ошибку среднего (стандартное отклонение / корень из числа наблюдений);

5. вычислите половину ширины интервала, используя стандартную ошибку среднего и критическое значение t -статистики;

6. определите доверительный интервал как среднее значение плюс-минус половина ширины интервала.

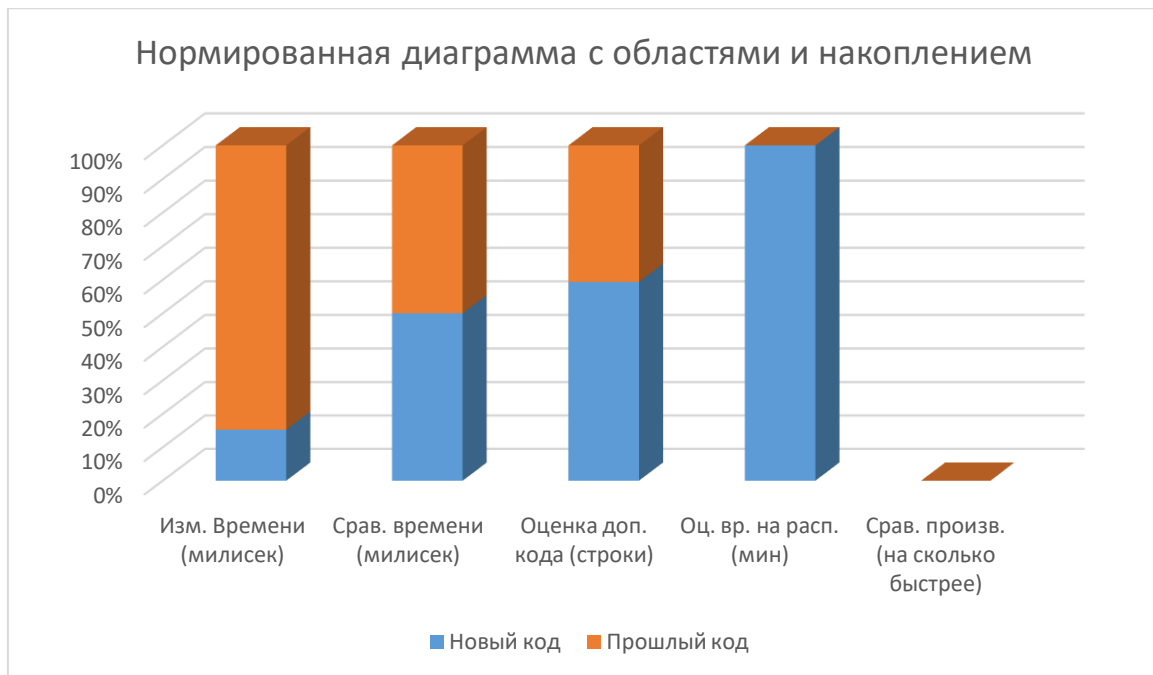
Необходимо посчитать время двумя способами, с помощью profiling и с помощью обычного замера.

Посчитанное время можно увидеть при выводе программы на рисунке 2.

Нужно оценить накладные расходы, такие как доля времени, проводимого на каждом этапе вычисления («нормированная диаграмма с областями и накоплением»), число строк кода, добавленных при распараллеливании, а также грубая оценка времени, потраченного на распараллеливание (накладные расходы программиста), и т.п. Все расчёты представлены в таблице 1.

Таблица 1 – Накладные расходы

	Изм. Времени (миллисек)	Срав. времени (миллисек)	Оценка доп. кода (строки)	Оц. вр. на расп. (мин)	Срав. произв. (на сколько быстрее)
Прошлый код	5000	4100	111	0	0
Новый код	900	4100	162	180	5.5



Таким образом, мы видим, что, прошлый код значительно уступает новому, потому что, он в 5 раз медленнее, и разница во времени вычислений тут очевидна. Хотя в новом коде больше строчек, но окупается скоростью и качеством вычислений на CUDA.

Даже с учётом того, что время на распараллеливание ушло так много, это не мешает новому коду хорошо выполнять свою задачу, и это очень сильно окупает силы программиста в данном случае.

ЗАКЛЮЧЕНИЕ

При использовании CUDA удалось добиться почти пятикратного ускорения выполнения программы.

Использование CUDA по сравнению с простым распараллеливанием с использованием общих механизмов многопоточности имеет несколько преимуществ, что делает его предпочтительным в определенных сценариях.

CUDA позволяет максимально эффективно использовать вычислительные ресурсы GPU. Это достигается благодаря возможности запуска большого количества параллельных вычислений на множестве ядер GPU.

CUDA предоставляет доступ к специализированным инструкциям и оптимизациям, предназначенным для работы с архитектурой GPU. Это может привести к значительному увеличению производительности по сравнению с общими механизмами многопоточности.

Таким образом, использование CUDA представляет собой мощный и эффективный способ реализации параллельных вычислений на GPU, что делает его предпочтительным выбором для широкого спектра задач, требующих высокой производительности и параллелизма.